

Università degli Studi di Salerno
Corso di Ingegneria del Software

MediBook
SDD
Versione 1.9



Data: 24/11/2025

Progetto: MediBook	Versione: 1.8
Documento: RAD	Data: 24/11/2025

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Lelio Crisci
--------------------	--------------

Revision History

Data	Versione	Descrizione	Autore
24/11/2025	1.1	Aggiunta punto 2 relativo al "Current software architecture"	Samuele Colucci
24/11/2025	1.2	Aggiunta punto 1 relativo all' "Introduction"	Lelio Crisci
24/11/2025	1.3	Aggiunta punto 3.3 relativo ad "Hardware Software Mapping"	Samuele Colucci
24/11/2025	1.4	Aggiunta punto 3.5 relativo ad "Access control and security"	Lelio Crisci
24/11/2025	1.5	Aggiunta punto 3.2 relativo a "Subsystem decomposition"	Christian Casale
24/11/2025	1.6	Aggiunta punto 3.7 relativo a "Boundary conditions"	Lelio Crisci
24/11/2025	1.7	Aggiunto punto 3.1 relativo a "Overview"	Christian Casale
24/11/2025	1.8	Aggiunto punto 3.4 relativo a "Persistent data management"	Christian Casale
24/11/2025	1.9	Aggiunto punto 4 relativo a "Subsystem services Glossary"	Lelio Crisci

Contents

1.	Introduction	4
1.1	Purpose of the system	4
1.2	Design goals	4
1.3	Definitions, acronyms, and abbreviations	5
1.4	References	5
1.5	Overview	6
2.	Current software architecture	7
3.	Proposed software architecture.....	7
3.1	Overview	8
3.2	Subsystem decomposition	9
3.3	Hardware/software mapping	10
3.4	Persistent data management	11
3.5	Access control and security	13
3.6	Global software control	14
3.7	Boundary conditions	14
4.	Subsystem services Glossary.....	16

1. Introduction

1.1 Purpose of the system

Lo scopo del sistema **MediBook** è la realizzazione di una piattaforma web centralizzata dedicata alla gestione e alla prenotazione delle visite mediche. Il sistema è progettato per sostituire l'attuale gestione manuale (telefonica o di persona), risolvendo criticità quali lunghi tempi di attesa, sovraccarico del personale amministrativo e difficoltà di coordinamento.

MediBook automatizza l'intero flusso di lavoro, permettendo ai pazienti di prenotare visite autonomamente, alla segreteria di gestire gli appuntamenti tramite dashboard dedicate e ai medici di gestire la propria agenda e i referti digitali. L'obiettivo finale è ottimizzare il processo, migliorare la qualità del servizio e garantire la tracciabilità delle operazioni.

1.2 Design goals

Illustriamo nella seguente tabella gli obiettivi di design per il sistema e le relative priorità (alta, bassa e media). Per ogni obiettivo riportiamo anche l'origine, facendo riferimento, in particolare, all'identificativo del requisito non funzionale ad esso associato.

Priorità	ID	Descrizione	Categoria	Origine
Bassa	DG_1	Il sistema deve garantire tempi di risposta non superiori a 3 secondi per le operazioni critiche (login, prenotazione, modifica)	Prestazioni (Performance)	NFR1
Media	DG_2	L'interfaccia utente deve essere intuitiva, permettendo di raggiungere le funzionalità principali in massimo 3 click.	Usabilità (Usability)	NFR2
Bassa	DG_3	L'architettura deve prevedere meccanismi di recovery che permettano il ripristino del sistema entro 2 ore in caso di guasto critico.	Affidabilità (Reliability)	NFR3
Alta	DG_4	Il sistema deve supportare almeno 200 utenti concorrenti simultanei senza degradare le prestazioni operative.	Scalabilità (Scalability)	NFR4
Media	DG_5	Il codice deve essere modulare e ben documentato (comprensibile in 3 giorni) e rispettare lo stack tecnologico definito (Java, ReactJS, MySQL).	Manutenibilità (Maintainability)	NFR5

Priorità	ID	Descrizione	Categoria	Origine
Media	DG_6	Il sistema deve garantire l'interoperabilità tramite formati standard (JSON, CSV, PDF) e compatibilità cross-browser (Chrome, Edge, Firefox, Safari).	Interfaccia (Inter)	NFR6
Bassa	DG_7	Devono essere disponibili strumenti di logging e monitoraggio per la gestione operativa e la rilevazione di errori da parte della segreteria.	Operazioni (Manageability)	NFR7
Alta	DG_8	Il sistema deve assicurare la conformità al GDPR e garantire la tracciabilità (audit log) di tutte le operazioni sensibili.	Sicurezza (Security)	NFR8

1.3 Definitions, acronyms, and abbreviations

- SSD: System Design Document.
- GDPR: General Data Protection Regulation.
- DG: Design Goal.
- RAD: Requirements Analysis Document.
- RNF: Requisiti Non Funzionali.

1.4 References

- Bernd Bruegge & Allen H. Dutoit, Object-Oriented Software Engineering Using UML, Patterns, and Java, 3rd edition.
- Slide del corso di Ingegneria del Software (Prof. De Lucia).
- Documento RAD.
- Requisiti non funzionali: Sezione 3.3 del RAD

1.5 Overview

- Il primo capitolo contiene l'obiettivo del sistema, la definizione dei design goal e una sezione dedicata agli acronimi per una maggiore chiarezza nella lettura della documentazione.
- Il secondo capitolo contiene una panoramica sull'architettura dei sistemi simili.
- Il terzo capitolo contiene la panoramica sull'architettura del sistema proposto, la decomposizione in sottosistemi, la mappatura hardware/software, la gestione dei dati persistenti, il controllo degli accessi, il controllo del flusso globale del sistema e le condizioni limite
- Il quarto capitolo contiene i servizi offerti dai sottosistemi.

2. Current software architecture

Attualmente, non esiste un prodotto software dedicato per la gestione degli appuntamenti e delle visite mediche nel contesto del progetto MediBook. L'architettura del sistema attuale è di natura manuale, basata sull'interazione telefonica, la comunicazione di persona e la gestione delle agende tramite registri cartacei o strumenti non digitali.

Questo processo operativo manuale è il sistema che MediBook si propone di sostituire, in quanto presenta notevoli inefficienze e problematiche:

- **Inefficienza:** Comporta lunghi tempi di attesa per i pazienti e un elevato sovraccarico del personale amministrativo (Segreteria).
- **Rischio di Errore:** La gestione manuale degli slot orari e delle comunicazioni aumenta il rischio di errori di pianificazione e incoerenze nei dati.

Questo sistema manuale deve essere sostituito perché causa inefficienze (tempi lunghi, sovraccarico amministrativo) e un alto rischio di errori.

Per definire le assunzioni e le scelte di design è stata condotta un'analisi su una piattaforma di successo nel medesimo dominio: [MioDottore](#).

MioDottore è una piattaforma di prenotazione di servizi sanitari accessibile via web e mobile. Dall'analisi della sua struttura e delle sue funzionalità, deduciamo i seguenti principi architetturali fondamentali, che MediBook intende adottare:

- **MVC:** Utilizzo il pattern architetturale MVC essendo quello maggiormente utilizzato per lo sviluppo di applicazioni web.
- **Gestione Dati Persistenti:** È fondamentale la presenza di un Database (DBMS) per la memorizzazione sicura, l'integrità e la gestione transazionale dei dati critici (es. dati dei Pazienti, Referti, Agende).
- **Controllo degli Accessi e Sicurezza:** L'accesso alle funzionalità è garantito da un meccanismo di autenticazione (username e password) per differenziare i ruoli (Paziente, Medico, Segreteria) e assicurare che i servizi siano fruibili solo da utenti autorizzati. La gestione dei dati sensibili deve essere conforme a normative legali (es. GDPR).
- **Accessibilità Web:** L'accesso multiplatforma (PC, tablet, smartphone) tramite browser web è standard per massimizzare l'usabilità e la disponibilità.

3. Proposed software architecture

3.1 Overview

Il sistema MediBook è concepito come una Web Application che adotta un'architettura a tre strati logici (Three-Tier Architecture), allo scopo di ottenere la massima separazione delle responsabilità, modularità e scalabilità (NFR4).

L'architettura utilizzata è basata su pattern architetturali Model-View-Controller (MVC) e Data Access Object (DAO), dove il Model è logicamente suddiviso tra gli strati di Business e di Persistence.

1. **Presentation Layer** (Interfaccia Utente)
Scopo: Gestire interazione con l'utente (Pazienti, Medici, Segreteria e Visitatori), la visualizzazione dei dati e definire il flusso la navigazione dell'applicazione.
2. **Business Layer** (Logica Applicativa)
Scopo: Definire e implementare la logica di business fondamentale dell'applicazione. Questo strato è responsabile della validazione dei dati, dell'elaborazione delle richieste (es. verifica disponibilità, aggiornamento agenda) e del coordinamento delle operazioni.
3. **Persistence Layer** (Gestione Dati Persistenti)
Scopo: Gestire esclusivamente il salvataggio, il recupero e la manipolazione dei dati persistenti, isolando il Business Layer dai dettagli specifici del Database Management System (DBMS). Questo strato garantisce la sicurezza e l'integrità dei dati (NFR8).

3.2 Subsystem decomposition

Il sistema è stato suddiviso nei seguenti sottosistemi:

Nella componente View:

- GUI Paziente: interfaccia utente dedicata al paziente.
- GUI Medico: interfaccia utente dedicata al medico
- GUI Segreteria: interfaccia utente dedicata alla segreteria

Nella componente Controller:

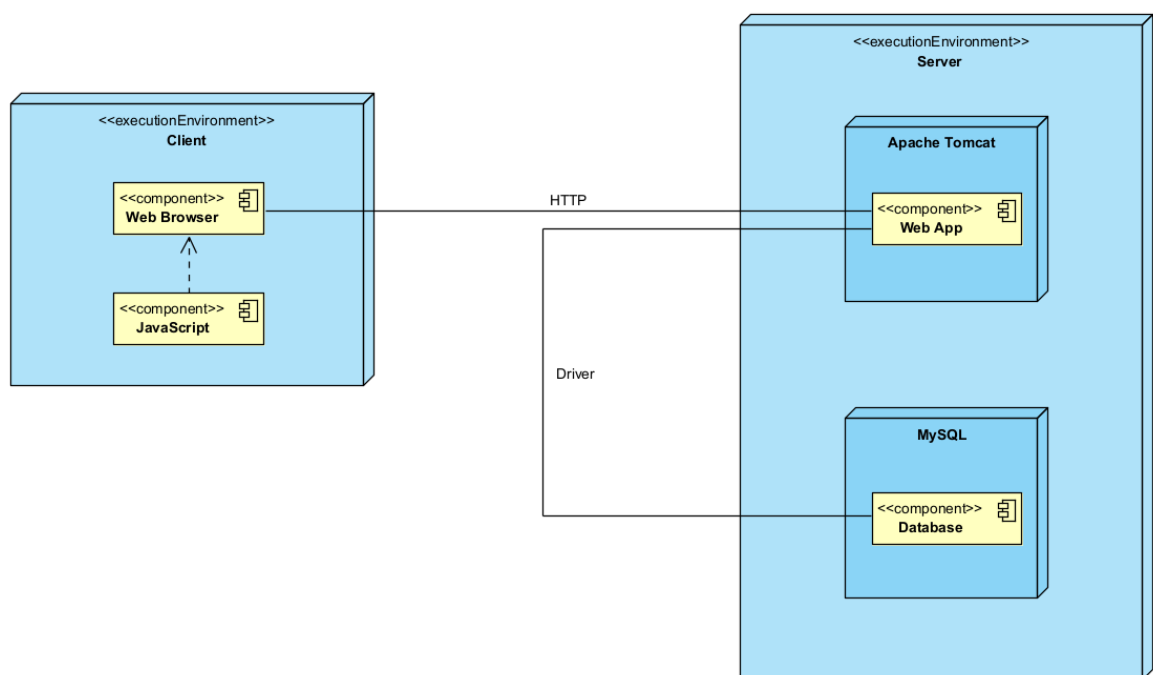
- Gestione utenza: si occupa delle operazioni di autenticazione.
- Gestione profilo: si occupa delle operazioni del proprio account.
- Gestione prenotazioni: si occupa di operazioni di gestione delle operazioni.
- Operazioni Medico: si occupa della parte di aggiornamento delle visite effettuate di un paziente.
- Operazioni Segreteria: si occupa della gestione delle prenotazioni.
- Gestione Prenotazione: Si occupa della logica di prenotazione, della verifica delle disponibilità e della gestione dello stato degli appuntamenti.
- Gestione Visite e Referti: Questo componente gestisce le azioni che avvengono dopo l'appuntamento (tipicamente eseguite dal Medico).

Nella componente Model:

- Gestione Dati: si occupa del salvataggio e del recupero delle informazioni mantenute nel Database.

3.3 Hardware/software mapping

Il sistema MediBook è implementato su un'architettura Client/Server a tre livelli (3-Tier) che separa l'interfaccia utente, la logica applicativa e la gestione dei dati. Il nodo Client è qualsiasi dispositivo che utilizza un browser web compatibile per inviare richieste al Server tramite HTTP. Il nodo Application Server (ambiente Linux con Java 17+ e Apache Tomcat) esegue tutta la logica di business. Su questo stesso nodo, o su un nodo dedicato logicamente separato, risiede il database relazionale MySQL 8.0, che gestisce lo storage dei dati persistenti (incluse agende e referti). L'Application Server interagisce con MySQL tramite driver specifici (come JDBC per Java) per garantire l'accesso rapido e transazionale ai dati, minimizzando i ritardi di rete grazie al posizionamento strategico dei componenti del Backend e assicurando che siano rispettati i requisiti di sicurezza (GDPR) e di scalabilità per gli utenti concorrenti.



3.4 Persistent data management

Per la gestione dei dati persistenti del sistema MediBook, si è scelto di utilizzare un Database Relazionale basato sui principi dell'algebra relazionale, dove le informazioni sono organizzate in tabelle bidimensionali. Il sistema utilizzerà MySQL 8.0 come DBMS, come definito nei requisiti di implementazione (NFR5).

Come risultato della fase di analisi, di seguito vengono riportati gli oggetti che dovranno essere memorizzati in maniera persistente:

Utente

Nome	Tipo
Username	String
Password	String
Email	String

Paziente

Nome	Tipo
Cartella clinica	Lista di file
Codice fiscale	String
Nome	String
Cognome	String

Segreteria

Nome	Tipo
Ruolo	String
Id	String

Medico

Nome	Tipo
Specialistica	String
Nome	String
Cognome	String

Prenotazione

Nome	Tipo
Id	String
Data prenotazione	Date
Stato prenotazione	String

Visita

Nome	Tipo
Id	String
Tipologia visita	String
Stato visita	String
Luogo visita	String

Referto

Nome	Tipo
Data caricamento	Date
Id	String
Contenuto	String

Agenda medico

Nome	Tipo
Calendario	Date

3.5 Access control and security

MediBook garantisce il controllo degli accessi mediante un meccanismo di autenticazione basato su email e password. Solo gli utenti registrati e autenticati possono usufruire dei servizi specifici associati al loro ruolo (Paziente, Medico, Segreteria) e accedere alle aree riservate per la gestione delle prenotazioni e dei dati clinici.

Come si evince dai Design Goals (Par. 1.2) e dal requisito NFR8 (Legal), la sicurezza e la conformità al GDPR sono priorità assolute per MediBook. Pertanto, il sistema implementa controlli rigorosi sugli input durante la fase di registrazione e login. Per garantire la riservatezza delle credenziali, tutte le password salvate nel database non sono memorizzate in chiaro, ma vengono crittografate tramite un algoritmo di hash sicuro. Inoltre, il sistema garantisce la tracciabilità delle operazioni effettuate dal personale medico e dagli utenti per motivi di audit.

Di seguito viene proposta una tabella dove sono elencate le operazioni che gli utenti possono effettuare in base al loro livello di accesso:

Attore	Livello di Accesso	Operazioni Consentite
Visitatore	Non autenticato	• Ricerca dei medici • Visualizzazione disponibilità medici (lettura) • Registrazione al sistema
Paziente	Autenticato	• Prenotazione visite • Visualizzazione storico e stato prenotazioni. • Gestione del profilo
Strategia	Autenticato	• Accesso alla Dashboard appuntamenti • Visualizzazione elenco prenotazioni • Modifica appuntamenti
Medico	Autenticato	• Consultazione agenda personale • Visualizzazione dettagli paziente • Gestione esito visita (Effettuata/Annulata) • Caricamento e gestione referti digitali

3.6 Global software control

Il controllo del flusso software globale di MediBook è gestito dal web server Apache Tomcat. Per ogni utente che si connette e invia una richiesta Tomcat crea un thread dedicato. Questo thread è responsabile di gestire la richiesta specifica, di coordinare le chiamate procedurali ai vari sottosistemi e di rispondere agli eventi generati. È fondamentale, tuttavia, tenere conto delle sezioni critiche che si verificano quando più thread tentano di modificare dati condivisi. Per garantire la consistenza dei dati, il sistema impiega la gestione transazionale e meccanismi di locking a livello del database, assicurando che le operazioni siano atomiche e isolate.

3.7 Boundary conditions

Questa sezione analizza il comportamento del sistema MediBook nelle fasi critiche del ciclo di vita operativo (avvio e arresto) e descrive le strategie di gestione in caso di malfunzionamenti o anomalie.

Startup

La fase di startup copre l'intervallo di tempo che intercorre tra l'accensione del server e la disponibilità del servizio agli utenti finali. L'inizializzazione del sistema avviene all'avvio del container web Apache Tomcat e prevede le seguenti attività sequenziali critiche:

1. **Caricamento della Configurazione:** Il sistema carica i parametri globali (es. credenziali DB, chiavi crittografiche, path di storage) dai file di configurazione sicuri.
2. **Inizializzazione del Connection Pool:** Il back-end Java tenta di stabilire un pool di connessioni persistenti verso il database **MySQL 8.0**. Se il database non è raggiungibile, la procedura di avvio viene interrotta e viene generato un log di errore critico.
3. **Verifica delle Risorse Esterne:** Viene verificata l'accessibilità e i permessi di scrittura sul File System del server Linux per la gestione dello storage dei referti medici PDF.
4. **Apertura del Servizio:** Solo se tutti i check precedenti hanno esito positivo, il sistema apre le porte HTTPS e inizia ad accettare le richieste degli utenti (Login, Prenotazione, ecc.).

Terminazione

La terminazione del sistema (shutdown) deve avvenire in modo controllato per garantire la consistenza dei dati, specialmente per le operazioni transazionali come le prenotazioni.

1. **Chiusura delle Sessioni:** Il sistema invalida tutte le sessioni utente attive, forzando il logout e salvando eventuali stati pendenti non critici.
2. **Completamento Transazioni:** Il sistema attende il completamento delle transazioni atomiche in corso sul database (es. conferma di una prenotazione). Se l'attesa supera un timeout predefinito, viene eseguito il rollback per garantire l'integrità dei dati.

3. **Rilascio Risorse:** Vengono chiuse le connessioni al database MySQL e rilasciati gli handle ai file dei referti aperti, prevenendo corruzioni o lock sui file.

Fallimento (Gestione dei guasti)

Il sistema è progettato per gestire condizioni di errore impreviste garantendo la continuità operativa o un degrado controllato, in conformità con i Requisiti Non Funzionali (NFR).

- **System Crash e Recovery:** In caso di arresto anomalo (crash hardware o software), il sistema deve poter essere ripristinato e tornare pienamente operativo entro un tempo massimo di **2 ore**, come definito nel requisito di affidabilità **NFR3**. Questo è garantito dalle procedure di backup automatico del database MySQL.
- **Sovraccarico (Overload):** Se il numero di utenti concorrenti supera la soglia di **200 utenti** (limite definito in **NFR4**), il sistema non deve collassare ma può accodare le richieste in eccesso, accettando un temporaneo aumento dei tempi di risposta oltre i 3 secondi previsti.
- **Gestione Eccezioni Dati:** In caso di errori durante la manipolazione dei dati (es. tentativo di prenotare uno slot già occupato da un'altra transazione concorrente), il sistema deve intercettare l'eccezione, eseguire il rollback della transazione e notificare l'utente con un messaggio di errore leggibile, senza esporre stack trace tecnici.

4. Subsystem services Glossary

Questa sezione descrive i servizi forniti dai sottosistemi identificati nella decomposizione del sistema. Ogni sottosistema espone un'interfaccia pubblica che permette agli altri componenti di accedere alle funzionalità principali di MediBook.

Sottosistema Gestione Utenti

Servizio	Descrizione
Autenticazione	Il sottosistema permette all'utente di autenticarsi tramite email e password, verificando le credenziali
Logout	Il sottosistema invalida la sessione dell'utente, impedendo ulteriori accessi fino a una nuova autenticazione
Registrazione	Il sottosistema permette di creare un nuovo account, creando un nuovo record utente nel database, con email e password univoci.
Area Personale	Il sottosistema recupera le informazioni del profilo dell'utente
Recupero Password	Il sottosistema gestisce le informazioni di recupero password

Sottosistema Appuntamenti

Servizio	Descrizione
Ricerca Dottore	Il sottosistema permette di ricercare i medici e le loro fasce orarie disponibili, in base alla specializzazione o il nome
Prenota Visita	Il sottosistema permette la registrazione di una nuova prenotazione
Modifica/Annulla Prenotazione	Il sottosistema permette di modificare o annullare una prenotazione non ancora svolta
Visualizza Elenco Prenotazioni	Il sottosistema recupera le informazioni delle prenotazioni di un utente

Sottosistema Referti e Visite

Servizio	Descrizione
Aggiorna Stato Visita	Il sottosistema permette di aggiornare lo stato di una visita contrassegnandola come effettuata/annullata/programmata
Carica Referto	Il sottosistema permette la registrazione di un nuovo referto in merito ad una visita
Visualizza Dettagli Visita/Referto	Il sottosistema permette di visualizzare le informazioni associate ad una visita o un referto
Storico Visite Paziente	Il sottosistema permette di recuperare le informazioni riguardante le visite effettuate da un paziente in passato